

SE 216 – SOFTWARE PROJECT MANAGEMENT
SOFTWARE MEASUREMENTS DOCUMENT

PROJECT NAME: REPLATE

GROUP NUMBER and MEMBERS: GROUP 7: Aylin DÖNMEZ, Beyza Makbule YILDIRIM, Eren PAMUK, Hayrünisa TOPRAK, Hüseyin Alp ERCANDANOĞLU, Hüseyin ESEN, Ramazan YILDIRIM

Questions to identify measurements:

- 1- How effectively is the platform engaging users and reducing overall food waste?**
- 2- How fast and reliable is the food reservation and handover workflow for citizens?**
- 3- How stable and responsive is the system under concurrent operations and external API integrations?**
- 4- How efficient is the team's agile development process and what is the resulting code quality?**

Identified measurements:

1. Measurement: Rescue Volume

- **Definition:** This measurement tracks the total amount of food (in portions or kg) saved from wasting to evaluate system efficiency.
- **Method:** The PostgreSQL database will aggregate the quantities of all successfully completed food reservations on a daily, weekly, and monthly basis.

2. Measurement: Restaurant Activity Rate

- **Definition:** This metric assesses user engagement by tracking the percentage of registered restaurants that regularly post food listings.
- **Method:** The Spring Boot backend will divide the number of restaurants that created at least one listing in a week by the total number of registered restaurants in PostgreSQL.

3. Measurement: Food Access Cycle Time

- **Definition:** This measurement monitors the average time elapsed between a citizen making a reservation and successfully collecting the food.

SE 216 – SOFTWARE PROJECT MANAGEMENT

SOFTWARE MEASUREMENTS DOCUMENT

- **Method:** The Spring Boot backend will calculate the time difference between the "reservation confirmed" timestamp and the "QR code verified" timestamp for each order.

4. Measurement: QR Code Verification Success Rate

- **Definition:** This measurement evaluates the reliability of the QR code scanning feature during the food handover process.
- **Method:** The Spring Boot backend will automatically calculate the ratio of successful, verified scans to the total number of scan attempts in the database.

5. Measurement: Concurrent Reservation Conflict Rate

- **Definition:** This measurement monitors the frequency of database errors occurring when multiple users attempt to reserve the same food listing simultaneously.
- **Method:** The Spring Boot server logs will be analyzed weekly to count the number of failed transactions and PostgreSQL rollbacks caused by concurrent booking attempts.

6. Measurement: API Response Latency

- **Definition:** This metric measures the average response time of the Google Maps API to ensure external integration does not degrade system performance.
- **Method:** Spring Boot Actuator (APM) will automatically measure the millisecond delay for every external Google Maps API request, and these metrics will be stored in Prometheus and visualized using Grafana.

7. Measurement: Sprint Velocity Achievement

- **Definition:** This metric measures the team's process efficiency by comparing the planned Story Points against the actually completed Story Points per Sprint.
- **Method:** The team will divide the completed Story Points by the initially committed Story Points mapped in GitHub Projects at the end of each Sprint review.

SE 216 – SOFTWARE PROJECT MANAGEMENT

SOFTWARE MEASUREMENTS DOCUMENT

8. Measurement: Defect Density

- **Definition:** This measurement tracks the critical bug density encountered during the development process to evaluate software quality.
- **Method:** The team will divide the total number of bugs logged in GitHub Issues by the total lines of code at the end of each Sprint.

Measurement storage and collection:

1. Measurement: Rescue Volume

- **What:** The total quantity of food (portions or kg) successfully rescued.
- **When:** Continuously updated in real-time, aggregated daily and weekly.
- **Format:** Integer or decimal data.
- **How:** Captured automatically by the Spring Boot backend whenever a food handover is successfully completed, and stored in the PostgreSQL database.

2. Measurement: Restaurant Activity Rate

- **What:** The percentage of registered restaurants that are actively creating listings.
- **When:** Calculated at the end of each week.
- **Format:** Percentage.
- **How:** A scheduled Spring Boot backend task runs weekly to compare the number of restaurants with new listings against the total number of registered restaurants, saving the result in the PostgreSQL database.

3. Measurement: Food Access Cycle Time

- **What:** The time elapsed between a citizen confirming a reservation and successfully verifying the QR code for pickup.
- **When:** Calculated per transaction upon successful pickup.
- **Format:** Time duration (Minutes/Hours).
- **How:** The Spring Boot backend calculates the difference between the "reservation timestamp" and the "QR verification timestamp" for each specific order ID, logging it in PostgreSQL.

4. Measurement: QR Code Verification Success Rate

SE 216 – SOFTWARE PROJECT MANAGEMENT

SOFTWARE MEASUREMENTS DOCUMENT

- **What:** The ratio of successful QR scans to total scan attempts.
- **When:** Logged continuously during runtime.
- **Format:** Percentage.
- **How:** The PostgreSQL database automatically logs every scan attempt (success or failure) triggered by the backend and calculates the overall success ratio.

5. Measurement: Concurrent Reservation Conflict Rate

- **What:** The number of transaction failures caused by users trying to book the same food simultaneously.
- **When:** Continuously monitored and reviewed at the end of each week.
- **Format:** Integer count logs.
- **How:** The Spring Boot application logs concurrent request failures and PostgreSQL database rollbacks whenever an overbooking (Optimistic Locking) attempt occurs, which are then analyzed weekly.

6. Measurement: API Response Latency

- **What:** The average response time of the Google Maps API integrations.
- **When:** Continuously logged during application usage.
- **Format:** Milliseconds (ms).
- **How:** Spring Boot Actuator automatically monitors the duration of every external Google Maps API request, which are then scraped and stored by Prometheus, and monitored through Grafana dashboards.

7. Measurement: Sprint Velocity Achievement

- **What:** The ratio of completed Story Points to the initially planned Story Points.
- **When:** At the end of every Sprint (during the Sprint Review).
- **Format:** Percentage.
- **How:** Recorded by the Scrum Master or development team in GitHub Projects to track and reflect the ratio of completed tasks and Story Points.

8. Measurement: Defect Density

- **What:** The number of confirmed bugs per thousand lines of code (KLOC).

SE 216 – SOFTWARE PROJECT MANAGEMENT

SOFTWARE MEASUREMENTS DOCUMENT

- **When: Evaluated at the end of every Sprint.**
- **Format: Decimal or ratio data.**
- **How: Reported and tracked using GitHub Issues by the development team, then calculated by dividing the total bug count by the current KLOC.**

Measurement Type	Description	Example Measurements
Rescue Volume	Tracks the total amount of food successfully saved from waste to evaluate the overall environmental and social impact of the system.	450 kg / 1,200 portions
Restaurant Activity Rate	Measures the engagement level by calculating the ratio of registered restaurants that actively post listings compared to the total number of registered businesses.	65% active
Food Access Cycle Time	Monitors the average duration between a citizen's initial reservation and the final successful collection of the food item.	42 minutes
QR Code Verification Success Rate	Calculates the percentage of successfully verified QR scans against the total scan attempts to evaluate the reliability of the food handover process.	94.5% success rate
Concurrent Reservation Conflict Rate	Monitors the frequency of database transaction failures or rollbacks that occur when multiple users attempt to book the exact same food listing simultaneously.	12 conflicts / 1,000 reservations (1.2%)
API Response Latency	Measures the average delay in milliseconds for Google Maps API requests to ensure external service dependencies do not degrade overall system performance.	245 ms average latency
Sprint Velocity Achievement	Measures the team's process efficiency by comparing the total number of Story Points actually completed against the Story Points initially planned for the Sprint.	85% (34/40 Story Points)
Defect Density	Tracks the critical bug density encountered during the development process to evaluate software quality and overall system stability.	3.2 bugs / KLOC